

# 5주차 늦게 도착한 이벤트를 셀 것인가 버릴 것인가

## 5주차 · 스트리밍 처리

이 장의 수치는 모두 `practice/chapter5/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

3절의 코드는 Spark 엔진을 띄우므로 Java 17과 Windows용 Hadoop 파일 두 개가 필요하다(2절 시작 전 준비). 이 판의 수치는 3절을 포함해 전부 그 준비를 마친 Windows 환경에서 지금 재실행해 얻은 값이다.

- ☐ 4주차는 이벤트를 유실 없이 받아 두는 데까지 왔고, 여기서 질문 하나를 남김
  - 22:04에 발생한 민원이 22:00~22:10 집계가 끝난 뒤에 도착하면 세야 하는가
- ☐ 이 장은 그 판단을 사람의 재량이 아니라 엔진이 따르는 규칙으로 바꿈
  - 규칙을 코드로 적고, 같은 입력에 그 규칙을 적용해 수용과 폐기가 어디에 증거를 남기는지 확인함

## 1. 오늘의 질문

- ☐ 끝이 없는 데이터에서 "최근 10분에 몇 건"을 세려면 어디를 잘라야 할까?
- ☐ 창을 겹치게 자르면 건수의 합은 무엇이 될까?
- ☐ 늦게 도착한 이벤트를 셀지 버릴지를 무엇이 정하고, 버린 건수는 어디서 확인할까?
- ☐ 집계 위에 알림을 얹으면 무엇이 새로 깨질까?

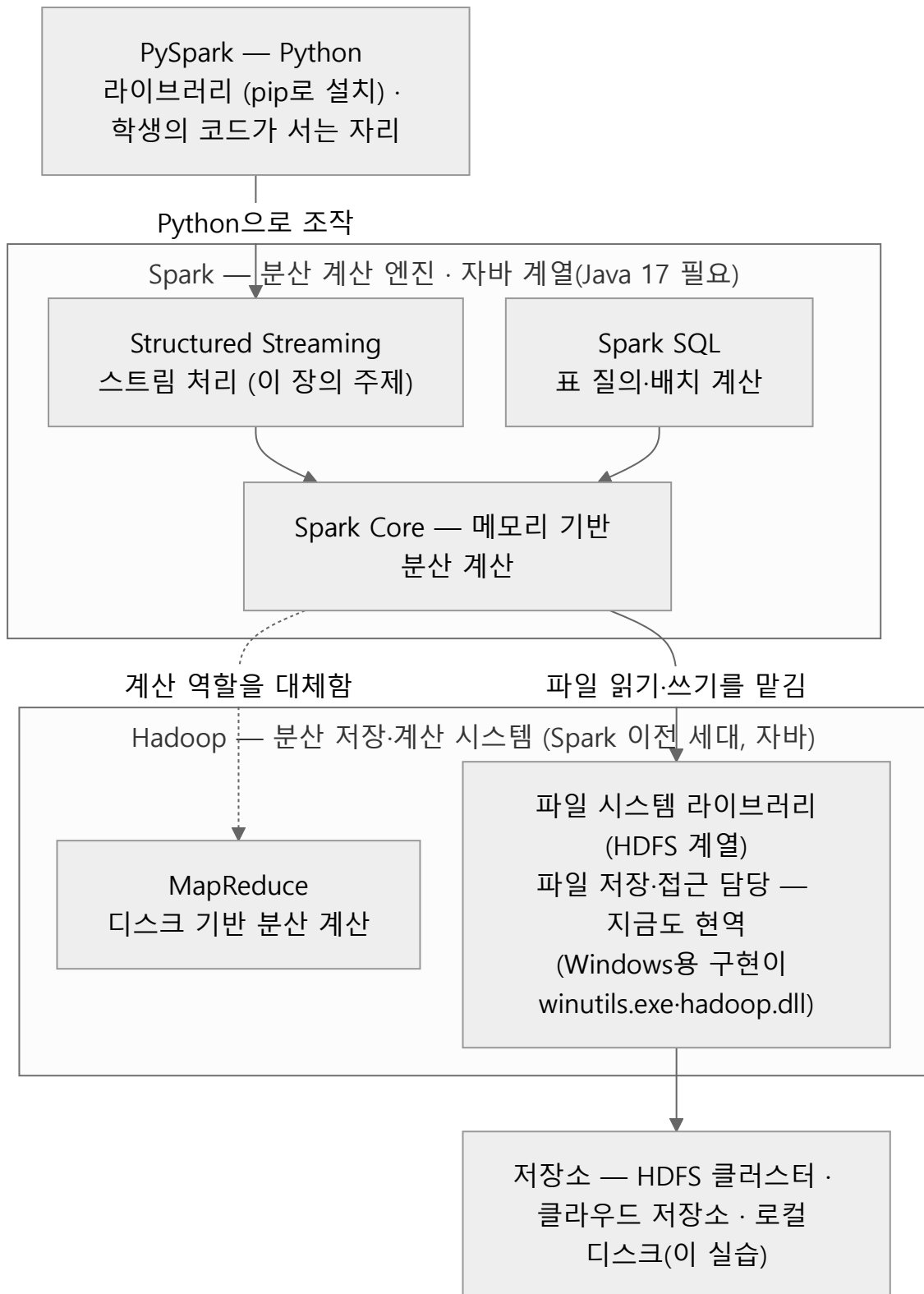
## 2. 이 장에서 실행할 것

### 용어부터 — 이 장에서 쓰는 낱말 열 개

- ☐ 각 용어는 앞에 나온 용어만 써서 정의함. 4주차에서 정의한 낱말(이벤트·브로커·토픽·생산자·소비자·처리)은 그대로 씀

- **스트림(stream)**: 끝을 정하지 않고 계속 이어지는 이벤트의 흐름이다. 4주차에서 Kafka 토픽에 쌓이던 민원 접수 이벤트가 그대로 이 장의 스트림이다.
- **Spark**: 큰 데이터를 여러 CPU와 서버에 나누어 계산하는 분산 처리 엔진이다. 한 컴퓨터로 몇 시간 걸릴 집계를 여러 대에 나눠 몇 분에 끝내는 용도로 쓰이며, 이 실습에서는 한 컴퓨터 안에서 CPU 2개로 돌린다. 자바 계열 언어로 만들어져 실행에 Java가 필요하다.

- **Hadoop(하둡):** Spark보다 먼저 나온 분산 처리 시스템으로, 큰 데이터를 여러 서버에 나누어 저장하는 부분과 계산하는 부분으로 이루어진다. 계산 부분은 Spark가 대체했지만, **파일을 읽고 쓰는 부분은 Spark가 지금도 Hadoop의 라이브러리를 빌려 쓴다.** 이 실습이 Windows에서 Hadoop의 Windows 전용 파일(winutils)을 요구하는 이유가 이것이다.
- **PySpark:** Spark 엔진을 Python 코드로 조작하는 라이브러리다. 실습에서 pip로 설치하는 것은 PySpark이고, Spark 엔진이 그 안에 함께 딸려 온다.
- **Structured Streaming:** Spark가 스트림을 처리하는 방식의 이름이다. 스트림을 "행이 계속 추가되는, 끝이 없는 표"로 간주하고 아는 표 질의를 그 위에 얹는다 — 자세한 동작이 3절의 주제다.
- **이벤트 시간(event time):** 이벤트가 실제로 발생한 시각이다. 민원이 접수된 시각이지, 그 이벤트가 처리 시스템에 도착한 시각이 아니다. 이 장의 집계는 전부 이벤트 시간 기준이다.
- **창(window):** 끝없는 스트림을 셀 수 있도록 시간축을 자른 구간이다. "22:00~22:10" 같은 10분 구간 하나가 창 하나다. 자르는 방식 두 가지(tumbling-sliding)는 4절에서 실행으로 비교한다.
- **상태(state):** 엔진이 창마다 들고 있는 중간 집계값이다. "창 22:00-22:10 강남구 3건"을 들고 있다가 한 건 더 도착하면 4로 고친다.
- **micro-batch:** 엔진이 스트림을 잘게 쪼개 처리하는 작은 배치 단위다. 새 데이터가 도착하면 그만큼을 한 번에 처리하고 다음을 기다린다.
- **watermark:** "지금까지 관측한 최대 이벤트 시간 - 지연 허용값"으로 계산되는 기준선이다. 이 선보다 오래된 이벤트는 더 오지 않는다고 간주하며, 늦게 도착한 이벤트를 셀지 버릴지를 이 선이 판정한다 — 5절의 주제다.



**그림 5.1** Spark와 Hadoop의 관계 — Hadoop은 큰 데이터를 여러 서버에 나누어 저장(HDFS)하고 계산(MapReduce)하던 이전 세대 시스템이고, Spark는 그중 **계산**을 메모리 기반으로 대체한 엔진이다. 대체되지 않고 남은 것이 **파일 저장·접근** 부분이라, Spark는 파일을 읽고 쓸 때 지금도 Hadoop의 파일 라이브러리를 부른다. 이 실습의 저장소는 클러스터가 아니라 로컬 디스크지만 그 접근에도 같은 라이브러리가 쓰이므로, Windows에서는 그 라이브러리의 Windows용 구현(winutils.exe-hadoop.dll)이 필요하다.

□ 이 장의 네 개념은 구청 민원실 창구의 하루에 하나씩 대응함

- 접수 대장의 줄이 하루 종일 늘어나듯 스트림은 행이 계속 추가되는 끝없는 표임
  - 시간대 칸으로 나누어 접수 건수를 세듯 이벤트를 시간 창으로 잘라 셈
  - 마감 시각에 곧바로 닫지 않고 규칙만큼 더 기다리듯 watermark가 지연 허용을 정함
  - 유예를 넘겨 온 서류를 접수하지 않고 반려 건수만 적듯 폐기된 이벤트는 카운터에만 남음
- 비유가 깨지는 지점은 마감 시각을 무엇이 정하느냐다
- 창구 직원은 벽시계를 보고 닫지만 watermark는 시계가 아니라 **지금까지 도착한 데이터**에서 계산됨
    - 이벤트가 하나도 들어오지 않으면 실제 시각이 아무리 흘러도 창은 닫히지 않음
  - 이 차이가 5절에서 숫자로 나타남
- 실행하기 전에 먼저 적음. 입력은 민원 이벤트 14건이고 창은 10분, 지연 허용은 10분임

**표 5.1** 예측표 — 실행하기 **전에** 채운다

| 예측할 것                                   | 내 예측 | 실제 | 어긋난 이유 |
|-----------------------------------------|------|----|--------|
| 강남구 7건을 10분 tumbling 창으로 세면 건수의 합       |      |    |        |
| 같은 7건을 10분 창·5분 슬라이드로 세면 건수의 합          |      |    |        |
| 22:04에 발생해 뒤늦게 도착한 이벤트의 판정(수용/폐기)       |      |    |        |
| 21:49에 발생해 뒤늦게 도착한 이벤트의 판정(수용/폐기)       |      |    |        |
| 이벤트 시각 범위에 걸친 10분 창 중 결과 행이 하나도 없는 창의 수 |      |    |        |

- 예측을 적지 않고 출력을 먼저 보면 거기 적힌 숫자가 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨
- 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

| 순서 | 실행할 파일                              | 하는 일                                        | 절 | 선행 조건                                |
|----|-------------------------------------|---------------------------------------------|---|--------------------------------------|
| 1  | 5-1-streaming-window-aggregation.py | Spark 스트리밍으로 창 집계<br>·watermark를 실제 엔진에서 돌림 | 3 | Java 17 · Windows는 winutils(시작 전 준비) |
| 2  | 5-2-window-assign.py                | 같은 이벤트를 tumbling과 sliding<br>창에 배정해 셈       | 4 | 없음                                   |
| 3  | 5-3-watermark-trace.py              | watermark 전진과 수용·폐기 판정을<br>재현하고 1번 결과와 대조   | 5 | 없음                                   |
| 4  | 5-4-alert-rules.py                  | 창 집계 위에 알림 규칙을 얹어 미확정<br>·중복·침묵을 관찰         | 6 | 없음                                   |

- 2·3·4번은 표준 라이브러리만 쓰므로 Python만 있으면 바로 돌아감
- 1번만 Spark 엔진을 띄움. 네 파일 모두 같은 입력( data/input/stream/batch1~3.json )을 읽음

## 시작 전 준비

- 1번을 실행하려면 PySpark가 필요하고, PySpark는 **Java 17 이상**이 있어야 됨 — 준비물 세 가지 (PySpark·Java·winutils)가 각각 어느 시스템의 것인지는 그림 5.1에 있음
  - 아래 한 줄이 Java 버전과 패키지를 점검하고 준비함

```
python scripts/setup_practice.py 5
```

- Java가 없거나 버전이 낮으면 위 명령이 **Temurin JDK 17 자동 설치를 시도함** (Windows는 winget, macOS는 Homebrew 사용)
  - 설치 도중 허용 여부를 묻는 창(UAC)이 뜨면 "예"를 누름. 설치에 수 분 걸림
  - 설치가 끝나면 PATH 반영을 위해 **새 터미널을 열고** 실습을 실행함
  - Windows에서 winget이 없거나 실패하면, 스크립트가 Temurin 공식 사이트(Adoptium)에서 설치 파일을 직접 내려받아 설치를 이어감
  - macOS에서 Homebrew가 없으면 안내 문구대로 Temurin JDK 17을 직접 설치함
- Windows에서는 파일 두 개를 더 준비함 — Spark가 로컬 파일을 읽을 때 부르는 Hadoop의 Windows 전용 파일 `winutils.exe` · `hadoop.dll`
  - `github.com/kontext-tech/winutils` 저장소의 `hadoop-3.4.0-win10-x64/bin` 에서 두 파일을 내려받아 `C:\hadoop\bin` 폴더에 둬
  - 환경 변수 `HADOOP_HOME` 을 `C:\hadoop` 으로 설정하고 PATH에 `C:\hadoop\bin` 을 추가한 뒤, **새 터미널**에서 실습을 실행함
  - 이 두 파일이 없으면 1번 실행이 `UnsatisfiedLinkError` 로 멈춤(3절 끝의 주의). macOS·리눅스는 이 준비가 필요 없음
- 2·3·4번은 이 준비 없이도 실행됨
  - 1번은 실행하는 동안 입력 디렉터리를 비웠다가 끝날 때 되돌림 — 중간에 실패해도 배치 파일 세 개가 그대로 남으므로 2·3·4번을 이어서 실행할 수 있음

## 3. 스트림은 끝없이 자라는 표다

- 도착하는 대로 처리하는 코드를 처음부터 새로 배우려면 부담이 큼
  - Spark Structured Streaming의 발상은 그 부담을 없애는 쪽임
  - 스트림을 "행이 계속 추가되는, 끝이 없는 테이블"로 간주하고 이미 아는 테이블 질의를 그 위에 그대로 얹는다
    - 배치에서 "지역구별 민원 건수"를 세던 질의를 고치지 않고 씀
- 달라지는 것은 실행 방식이다
  - 강남구 민원이 한 건 더 도착했을 때 엔진은 전체를 처음부터 다시 세지 않음
    - 이미 들고 있던 "강남구 3"이라는 중간 결과에 1을 더해 4로 고침

- 이 중간 결과를 **상태(state)** 라고 부른다
- 상태가 있어서 증분 계산이 되지만, 상태는 쌓이므로 "언제 버릴 것인가"라는 질문이 따라옴
  - 그 답이 5절의 watermark다
- 엔진은 흐름을 아주 잘게 쪼갠 작은 배치의 연속으로 처리하며, 이 단위를 **micro-batch**라고 한다
  - 4주차에 직접 돌렸던 폴링 루프(poll→처리→커밋)와 구조가 닮았고, 다만 루프를 사람이 아니라 엔진이 돌림
  - 검증된 배치 실행 엔진을 재사용하고 "이 배치를 처리했는가"를 단위로 실패를 복구할 수 있음
  - 대신 최소 지연이 배치 주기만큼 생김
    - 초에서 분 단위면 충분한 공공 대시보드에서는 감수할 만한 절충임
- micro-batch를 언제 돌릴지는 **trigger**가 정한다
  - 지정하지 않으면 앞 배치가 끝나는 대로 다음 배치를 돌려 지연을 최소화함
  - `processingTime="1 minute"` 으로 두면 1분마다 돌아 대시보드 갱신 주기에 맞춤
  - `availableNow` 는 쌓인 데이터를 모두 처리한 뒤 종료함
    - 스트리밍 코드로 정기 증분 작업을 구현하는 셈이 되어 6주차의 배치와 만남
- 무엇을 내보낼지는 **output mode**가 따로 정한다
  - complete는 결과 표 전체를 매번 내보내고, update는 이번 배치에서 바뀐 행만 내보냄
    - update 모드에서는 같은 키(창+지역구)의 행이 여러 번 나오므로 받는 쪽 저장소가 4주차의 업서트(upsert)로 받아야 함
  - append는 다시는 바뀌지 않을 확정 행만 내보냄
    - "이 창은 다시 안 바뀐다"고 판단할 근거가 필요하므로 watermark 없이는 집계에 쓸 수 없음
- 같은 스트림에 같은 질의를 걸어도 이 선택 하나로 산출물의 성격이 바뀜
  - 재난 상황판의 "지금까지의 지역별 신고 건수"는 계속 갱신되어야 하니 update나 complete임
  - 사후 감사용으로 남기는 "확정된 10분 단위 집계"는 불변 파일이어야 하니 append와 watermark를 함께 씀
  - 어떤 수치가 갱신되는 잠정치이고 어떤 수치가 확정 기록인지 산출물마다 적어 두지 않으면, 나중에 "대시보드와 보고서가 왜 다른가"라는 질문에 설계로 답할 수 없음
- 첫 번째 코드를 실행함. 파일 세 개를 스트림 입력 디렉터리에 차례로 도착시키고 update 모드로 출력함

```
cd practice/chapter5
python3 code/5-1-streaming-window-aggregation.py # Java 17 필요
cat data/output/ch5_window_counts.jsonl
```

```
# 출처: practice/chapter5/data/output/ch5_window_counts.jsonl
{"micro_batch": 0, "window_start": "22:00", "window_end": "22:10", "district": "마포구", "count": 2}
{"micro_batch": 0, "window_start": "22:00", "window_end": "22:10", "district": "관악구", "count": 1}
{"micro_batch": 0, "window_start": "22:00", "window_end": "22:10", "district": "강남구", "count": 3}
{"micro_batch": 2, "window_start": "22:10", "window_end": "22:20", "district": "강남구", "count": 2}
{"micro_batch": 2, "window_start": "22:10", "window_end": "22:20", "district": "마포구", "count": 2}
{"micro_batch": 2, "window_start": "22:00", "window_end": "22:10", "district": "강남구", "count": 4}
{"micro_batch": 4, "window_start": "22:20", "window_end": "22:30", "district": "관악구", "count": 1}
{"micro_batch": 4, "window_start": "22:20", "window_end": "22:30", "district": "강남구", "count": 1}
```

- 화면에 나온 값에서 읽을 것 — 앞에서 고른 output mode(update)가 각 줄에 어떻게 나타났는지를 봄
  - **바뀐 행만 나옴 — update 모드의 정의가 화면에 그대로 나타난 것임.** 여덟 줄이 전부이고, 배치 0이 만든 세 행은 배치 2·4에서 다시 나오지 않음. complete 모드였다면 매 배치마다 그때까지의 결과와 전체가 다시 나와 같은 행이 배치 수만큼 반복됐을 것임
  - **같은 키가 두 번 나오는 것은 update 모드의 대가임 —** 창 22:00–22:10의 강남구가 배치 0에서 3, 배치 2에서 4로 두 번 등장함. 받는 쪽이 이 둘을 각각 새 행으로 적으면 건수가 7로 부풀고, 창과 지역구를 키로 삼은 업서트로 받아야 4가 됨. 앞에서 "update는 받는 쪽이 업서트로 받아야 한다"고 한 것이 바로 이 두 줄 때문임
  - **append 모드였다면 이 시점에 거의 아무것도 안 나왔음 —** append는 다시는 안 바뀔 확정 행만 내보내므로 강남구 3처럼 나중에 바뀔 값은 아예 나오지 않고, "다시 안 바뀐다"는 판정 자체가 watermark의 창 닫힘(5절)을 기다림. 잠정치를 계속 갱신해 보여 주는 대시보드 용도라 update를 골랐던 것임
  - **배치 번호가 1, 3, 5를 건너뛴 —** 데이터를 처리한 배치가 watermark를 전진시키면, 엔진은 새 데이터가 없어도 전진한 기준선으로 상태를 정리하는 배치를 한 번 더 돌림. 이 빈 배치가 1·3·5번이라 micro-batch 번호와 입력 파일 번호는 일치하지 않음  
(practice/chapter5/data/output/ch5\_late\_event\_report.json의 micro\_batches에 빈 배치가 그대로 기록되어 있음)
  - **21:40–21:50 창은 어디에도 없음 —** 파일 3에 21:49에 발생한 이벤트가 들어 있는데 결과에는 그 창이 없음. 무슨 일이 있었는지는 집계 결과만 봐서는 알 수 없고, 5절에서 확인함

- **Windows에서는 Java 17만으로 부족하고 Hadoop 파일 두 개가 더 필요하다**
- Spark가 로컬 파일을 훑을 때 Hadoop의 Windows 전용 함수를 부르므로, winutils.exe · hadoop.dll 이 없으면 위 명령이 UnsatisfiedLinkError 로 멈춘다. 준비 방법은 2절 "시작 전 준비"에 있다.
- 이 절의 수치는 그 준비를 마친 Windows 11 환경(Python 3.13 · PySpark 4.2.0 · Temurin JDK 17 · hadoop-3.4.0용 winutils)에서 재실행해 얻은 값이다. 실행 기록은 practice/chapter5/results/5-1-streaming-window-aggregation.log 에 있다.

- 실행이 막히면 억지로 우회하지 말고 **어디서 막혔는지를 기록**한다. 이것도 운영에서 남겨야 하는 증거다.

## 4. 시간 창 — 겹치지 않게 자를까, 밀면서 겹칠까

- 무한 테이블에서 지역구별 건수를 그냥 세면 그 수는 시스템이 켜져 있는 한 계속 증가함
  - 운영에서 필요한 것은 "지금까지 전부"가 아니라 "최근 10분에 몇 건"이므로 이벤트를 시간 창으로 잘라 창 안에서 셈
  - 이때 기준 시각은 도착 시각이 아니라 **발생 시각(이벤트 시간)** 이어야 자연 도착한 민원도 발생한 시간대의 통계에 들어감
    - 도착 시각으로 세면 22:04에 발생한 민원이 22:10 이후 통계로 넘어가, 그 시간대에 실제로 무슨 일이 있었는지 되짚을 수 없게 됨
  - 그래서 이벤트에 발생 시각이 없거나 원천마다 시간대 표기가 다르면, 창 집계를 시작하기 전에 그것부터 맞춰야 함
- 창을 만드는 방식은 두 가지다
  - **tumbling window(구르는 창)** 는 시간축을 겹치지 않게 자름
    - 22:00–22:10과 22:10–22:20처럼 벽시계의 10분 칸을 그대로 쓰므로 모든 이벤트가 정확히 한 창에 속함
    - 정기 보고와 확정 통계에 맞음
  - **sliding window(미끄러지는 창)** 는 창 크기는 같지만 일정 간격으로 창을 밀어 겹치게 만듦
    - 크기 10분에 슬라이드 5분이면 21:55–22:05, 22:00–22:10, 22:05–22:15로 이어짐
    - 한 이벤트가 정확히 두 창에 속한다(창 크기 ÷ 슬라이드 = 2)
  - 코드에서 두 방식의 차이는 인자 하나이므로 실수로 바꿔 쓰기 쉬움

```
F.window("event_time", "10 minutes")           # tumbling: 슬라이드 없음
F.window("event_time", "10 minutes", "5 minutes") # sliding: 5분마다 새 창
```

- 두 번째 코드를 실행함. 같은 14건을 두 방식으로 세고 경계 규칙까지 확인함

```
python3 code/5-2-window-assign.py
```

# 출처: practice/chapter5/results/5-2-window-assign.log

[1] 입력 - 이벤트 14건, 파일 3개, 지역구 3개

이벤트 시각 범위 21:49 ~ 22:24

창 크기 10분, 슬라이드 5분, 기준 시각은 발생 시각(이벤트 시간)

[2] tumbling 10분 - 창끼리 겹치지 않음

21:40-21:50 마포구 1

22:00-22:10 강남구 4, 마포구 2, 관악구 1

22:10-22:20 강남구 2, 마포구 2

22:20-22:30 강남구 1, 관악구 1

[3] 강남구 7건을 두 방식으로 셈 (발생 22:01, 22:03, 22:04, 22:08, 22:11, 22:14, 22:24)

tumbling 22:00-22:10 → 4, 22:10-22:20 → 2, 22:20-22:30 → 1

건수 합 7

sliding 21:55-22:05 → 3, 22:00-22:10 → 4, 22:05-22:15 → 3, 22:10-22:20 → 2, 22:15-22:25 → 1, 22:20-22:30 → 1

건수 합 14

이벤트 수 7건 대비 tumbling 합 7, sliding 합 14 - 2배로 세어짐 (창 크기 ÷ 슬라이드 = 2)

[4] 경계 판정 - 창은 [시작, 끝)이다 (아래 세 시각은 입력에 없는 구성값)

22:09:59 → tumbling 22:00-22:10

22:10:00 → tumbling 22:10-22:20

22:10:01 → tumbling 22:10-22:20

#### □ 화면에 나온 값에서 읽을 것

- **tumbling의 합은 이벤트 수와 같음** — 강남구 7건이 창 셋에 4, 2, 1로 나뉘고 합이 7임. 한 이벤트가 한 창에만 들어가기 때문임
- **sliding의 합은 이벤트 수의 두 배임** — 같은 7건이 창 여섯에 흩어져 합이 14가 됨. 오류가 아니라 슬라이드가 창 크기의 절반이라 모든 이벤트가 정확히 두 번 세어진 결과임. 배수는 창 크기 ÷ 슬라이드이므로 슬라이드를 2분으로 줄이면 배수도 달라지며, 그 값은 9절의 실습으로 직접 확인함
- 그러므로 **sliding 창**의 건수를 창끼리 더한 수는 이벤트 수가 아님. 감사에서 "대시보드는 14건인데 보고서는 7건"이라는 질문을 받지 않으려면 집계표에 창 방식·크기·슬라이드를 함께 적어야 함
- **경계는 시작을 포함하고 끝을 포함하지 않음** — 22:09:59는 앞 창이고 22:10:00은 다음 창임. 손으로 계산한 값과 엔진 결과가 1건 어긋나는 흔한 원인이 이 규칙임
- **21:40-21:50 창에 마포구 1건이 있음** — 3절의 엔진 결과에는 없던 창임. 이 스크립트는 지연을 따지지 않고 배정만 하기 때문이며, 엔진에서 이 건이 어디로 갔는지는 다음 절에서 밝혀짐

#### □ 어느 방식을 쓰지는 그 수치로 무엇을 판단하는지가 정함

- 확정 통계와 정기 보고는 tumbling으로 냄

- 한 이벤트가 한 창에만 들어가므로 창별 건수를 더하면 그대로 총계가 되고, 같은 기간을 두 번 세지 않음
- 급증 탐지는 sliding이 유리함. "10분 안에 3건 이상이면 경보"라는 임계 규칙을 걸었다고 하고, 강남구에 실제로 몰린 세 건(22:08-22:11-22:14 — 6분 사이 3건)을 따라가 보면 차이가 드러남
  - tumbling에서는 창 경계 22:10이 이 뭉치의 한가운데를 지나가므로, 세 건이 앞 창에 1건(22:08)·뒤 창에 2건(22:11-22:14)으로 쪼개짐. 뭉치만 보면 어느 창도 3건이 못 되어, 몰림은 실재하는데 창 단위 건수로는 보이지 않음
  - sliding에는 그 경계를 걸치는 창 22:05-22:15가 있어 세 건을 통째로 담음. 위 실행 결과의 22:05-22:15 → 3 이 그 값이고, 이 창이 임계 3에 닿아 경보가 울림
  - 요컨대 tumbling은 자르는 위치가 고정되어 있어 하필 그 위치에 몰린 사건을 반으로 갈라 놓치지만, sliding은 자르는 위치를 5분마다 하나씩 더 만들어 두므로 어디에 몰리든 그 몰림을 통째로 담는 창이 하나는 있음
- 대신 sliding은 결과와 상태가 그만큼 늘어남
  - 같은 강남구 이벤트가 tumbling에서는 창 3개였는데 sliding에서는 창 6개가 되므로, 엔진이 들고 있어야 하는 창의 수도 두 배임

- 강남구 7건에는 뒤늦게 도착한 22:04가 들어 있다
- 이 절은 도착 순서를 무시하고 발생 시각만으로 배정한다. 그래서 22:04 이벤트가 22:00-22:10 창에 들어가 그 창이 4가 된다.
- 3절의 엔진 결과에서 같은 창이 처음 3으로 나왔다가 4로 갱신된 것은 도착 순서 때문이다. 두 숫자가 다른 것이 아니라 집계를 확정된 시점이 다르다.

## 5. watermark — 언제까지 기다릴지를 데이터로 정한다

- 이벤트 시간으로 집계하면 근본적인 긴장이 생김
  - 창을 빨리 확정하면 늦게 도착한 이벤트를 놓치고, 무한정 기다리면 아무것도 확정하지 못한 채 열린 창의 상태를 계속 들고 있어야 함
  - watermark는 이 긴장을 선언 한 줄로 타협함
- watermark는 관측된 최대 이벤트 시각에서 지연 허용값을 뺀 시각이다
  - "이보다 오래된 이벤트는 더 오지 않는다고 간주한다"는 기준선임
  - 창의 끝이 이 기준선보다 과거가 되면 엔진은 창을 닫고 상태를 정리한 뒤 그 창에 오는 지연 이벤트를 폐기함

```
events.withWatermark("event_time", "10 minutes")
    .groupBy(F.window("event_time", "10 minutes"), "district").count()
```

- 여기서 반드시 붙잡아야 할 원리는 계산의 기준이 시계가 아니라는 점임

- "지금 시각 - 10분"이 아니라 "지금까지 본 이벤트 중 가장 최근 것의 이벤트 시각 - 10분"임
- 데이터가 안 들어오면 watermark는 멈춰 있고, 데이터가 몰려 들어오면 빠르게 전진함
- 세 번째 코드를 실행함. 규칙 두 줄만 Python으로 구현해 파일이 도착할 때마다 watermark를 다시 계산함

python3 code/5-3-watermark-trace.py

# 출처: practice/chapter5/results/5-3-watermark-trace.log

[2] batch1.json 도착 - 이벤트 6건, 적용 watermark 없음(아직 관측한 이벤트 없음)

수용 6건 / 폐기 0건, 폐기 누적 0

처리 후 관측 최대 22:09 → 다음 watermark 21:59 (= 22:09 - 10분)

[3] batch2.json 도착 - 이벤트 5건, 적용 watermark 21:59

갱신 창 22:00-22:10 강남구 3 → 4

신규 창 22:10-22:20 강남구 0 → 1

수용 5건 / 폐기 0건, 폐기 누적 0

처리 후 관측 최대 22:18 → 다음 watermark 22:08 (= 22:18 - 10분)

[4] batch3.json 도착 - 이벤트 3건, 적용 watermark 22:08

폐기 LATE\_DROP 발생 21:49 - 창 21:40-21:50의 끝이 watermark 22:08 이하

수용 2건 / 폐기 1건, 폐기 누적 1

처리 후 관측 최대 22:24 → 다음 watermark 22:14 (= 22:24 - 10분)

[5] 자연 이벤트 판정

LATE\_OK 발생 22:04 도착 시점 watermark 21:59 창 22:00-22:10 → 수용

증거: 창 22:00-22:10 강남구 count 3 → 4

LATE\_DROP 발생 21:49 도착 시점 watermark 22:08 창 21:40-21:50 → 폐기

증거: 폐기 누적 = 1

[6] 가정 시나리오 - LATE\_OK가 마지막 파일 다음에 도착했다면 (입력에 없는 계산)

적용 watermark 22:14, 창 22:00-22:10의 끝 22:10 → 폐기

[7] 5-1(Spark) 산출물과 대조

엔진 판정 {'LATE\_OK': '수용', 'LATE\_DROP': '폐기'} / 이 스크립트 판정 {'LATE\_OK': '수용', 'LATE\_DROP': '폐기'}

엔진 폐기 누적 1 / 이 스크립트 폐기 누적 1

일치: 예 - 규칙을 손으로 적용한 결과와 엔진 결과가 같음

- 화면에 나온 값에서 읽을 것 — [2]→[4] 각각이 "직전 기준선으로 심사 → 창 끝과 비교해 수용·폐기 → 관측 최대로 기준선 전진"의 한 바퀴임
  - 관측 최대는 도착 시각이 아니라, 읽은 이벤트들의 발생 시각 중 최댓값임 — [3]에서 22:04가 섞여 와도 관측 최대는 22:18임. 늦은 이벤트가 최댓값을 끌어내리지 못하므로 watermark는 앞으로만 전진함
  - watermark는 거기서 지연 허용 10분을 뺀 값임([2]~[4]의 마지막 줄) — 지연 허용은 "최신 발생보다 10분까지 뒤쳐진 것은 받아 준다"는 약속이라, 마감선이 최신 발생의 과거 쪽에 그어짐. 그래서 더하지 않고 뺌
  - 수용은 창 갱신으로 보임([3]의 "3 → 4") — 심사 기준선은 직전 파일까지로 계산한 21:59였고, 창 끝 22:10이 그보다 미래라 창이 열려 있었음
  - 폐기는 카운터에만 남음([4]) — 창 21:40~21:50의 끝이 기준선 22:08보다 과거라 버려졌고, 집계에는 그 창의 행이 없을 뿐임
  - 판정은 도착 시점이 정함 — 같은 22:04라도 한 파일 늦게 왔다면 폐기였음([6]). 이 규칙을 손으로 적용한 결과는 엔진 산출물과 일치함([7])
- 창이 닫히는 조건도 같은 비교식임 — 창의 끝  $\leq$  watermark
  - 마지막 처리 후 watermark는 22:14이므로 끝이 22:20인 창 22:10~22:20은 아직 열려 있고, 파일이 더 오지 않으면 영원히 닫히지 않음
  - 3절의 append 모드가 watermark 없이 집계에 못 쓰이는 이유가 이 비교임 — "다시 안 바뀐다"의 판단 근거가 이것뿐이기 때문
- 수용과 폐기는 증거가 남는 곳이 다름
  - 수용은 집계 행의 갱신으로 보이지만 폐기는 행이 없을 뿐이므로, 폐기는 집계가 아니라 카운터로 감시함 — Spark의 `numRowsDroppedByWatermark` 메트릭이며 `ch5_late_event_report.json`에 파일별 누적 0-0-1로 기록됨
  - 4주차에서 커밋을 앞세운 시나리오의 유실이 오류 없이 조용했던 것과 같은 구조임
- watermark는 폐기 규칙이면서 상태 청소 규칙임
  - 지연 허용을 길게 잡을수록 늦은 이벤트를 더 받는 대신 모든 창의 상태를 그만큼 오래 들고 있어야 함 — "넉넉하게 하루로 잡자"는 공짜가 아님
- 지연 허용값은 실제 지연 분포(도착 시각 - 발생 시각)를 모아 그 꼬리를 덮도록 정하고, 운영 중 폐기 카운터가 늘면 분포가 변했다는 신호로 읽어 다시 계산함

## 6. 알림 — 집계 위의 필터에 함정이 셋

- 창 집계가 만들어지면 알림은 그 위에 얹는 필터임
  - "10분 창에서 같은 지역구 민원이 N건 이상이면 알린다"처럼 한 줄로 쓸 수 있음
  - 이때 경보를 울릴지 가르는 기준 건수 N을 임계값(threshold)이라 부름
  - 여기에 앞에서 본 문제가 모두 다시 등장하므로, 임계값을 4건과 3건으로 바꿔 가며 확인함
- 네 번째 코드를 실행함. update 모드의 갱신 흐름을 재현하고 그 위에 임계 규칙을 얹음

# 출처: practice/chapter5/results/5-4-alert-rules.log

[2] 임계 4건 - 갱신마다 조건을 다시 평가하면

발화 batch2.json에서 창 22:00-22:10 강남구 4건

발화 1회, 중복 제거 키(창 시작·지역구·규칙 ID)로 묶으면 1회 → 중복 0회

[3] 임계 3건 - 갱신마다 조건을 다시 평가하면

발화 batch1.json에서 창 22:00-22:10 강남구 3건

발화 batch2.json에서 창 22:00-22:10 강남구 4건

발화 2회, 중복 제거 키(창 시작·지역구·규칙 ID)로 묶으면 1회 → 중복 1회

[4] 발화 시점 - 창이 끝난 뒤에 도착한 이벤트로 임계에 닿는다

창 22:00-22:10 강남구 4건이 batch2.json 처리 시점에 임계에 닿음

창의 끝은 22:10, 확정까지 기다리면 이벤트 시간 기준 22:20 이후 - 확정 알림은 최소 10분 늦음

[5] 침묵 - 건수 0인 창은 0으로 나오지 않고 행 자체가 없다

이벤트 시각 범위 21:49~22:24에 걸친 10분 창을 모두 늘어놓으면

창 21:40-21:50 결과 행 없음

창 21:50-22:00 결과 행 없음

창 22:00-22:10 결과 행 3개 강남구 4, 관악구 1, 마포구 2

창 22:10-22:20 결과 행 2개 강남구 2, 마포구 2

창 22:20-22:30 결과 행 2개 강남구 1, 관악구 1

창 5개 중 결과 행이 없는 창 2개

#### □ 첫째 함정은 미확정 창에서 올리는 알림임

- 임계 4건으로 두면 파일 1을 처리한 시점에 강남구는 3이라 조용하다가, 파일 2에서 지연분이 수용되어 4가 되는 순간 발화함
- 그런데 그 시점은 창이 끝난 22:10보다 뒤임. 창이 확정된 뒤에 평가하기로 하면 판정이 뒤집히는 오경보는 없지만 **이벤트 시간 기준으로 최소 10분(= 지연 허용) 더 늦어짐**
- 빠른 알림과 정확한 알림은 같은 설계가 아니며, 무엇을 택했는지 적어 두지 않으면 나중에 설명할 수 없음

#### □ 둘째 함정은 알림 중복임

- 임계를 3건으로 낮추면 같은 창이 3에서 한 번, 4에서 또 한 번 발화해 2회가 됨
- 창 시작 시각과 지역구와 규칙 ID를 묶어 중복 제거 키로 삼으면 1회로 줄어듦
- 4주차에서 본 중복 문제가 알림 계층에서 그대로 재현된 것이고 해법도 같음
  - 발송 기록에 유니크 제약을 걸어 4주차의 맥등 쓰기를 적용함

#### □ 셋째 함정은 침묵임

- 알림 조건은 "데이터가 있는데 많음"만 잡아내므로 데이터가 아예 안 들어오는 수집 장애는 어떤 창도 임계를 넘지 않아 조용함
- 더 까다로운 점은 0건인 창이 건수 0인 행으로 나오지 않고 **행 자체가 없다는 것**임
  - 창 다섯 중 둘에 결과 행이 없는데, 하나(21:50-22:00)는 이벤트가 없어서이고 다른 하나(21:40-21:50)는 폐기되어서임
  - 집계 결과만 보면 둘을 구분할 수 없으므로 "0건이면 경보"라는 조건을 집계 위에 걸 수 없음
- 일정 시간 결과 행이 하나도 안 나왔음을 감시하는 하트비트(heartbeat)나 유입량 하한 감시를 따로 두어야 한다(11주차)
- 4주차의 유실, 5절의 폐기에 이어 세 번째로 확인하는 같은 교훈임

#### • 이 절의 발화는 계산이지 발송이 아니다

- `5-4-alert-rules.py` 는 임계 조건을 만족하는 갱신이 몇 번 나오는지를 셀 뿐이고 실제 알림을 보내지 않는다.
- 실제 발송 계층에는 재시도가 붙으므로 4주차의 중복 문제가 한 번 더 생긴다. 중복 제거 키를 규칙 계층에만 두면 발송 계층의 재시도는 막지 못한다.

#### □ 그래서 알림을 엮을 때 적어 두어야 하는 결정은 셋임

- 언제 평가하는가 — 갱신마다 평가할지, 창이 확정된 뒤에 평가할지
- 무엇으로 같은 알림을 묶는가 — 창 시작 시각·대상·규칙 ID를 묶은 키를 쓰지
- 조용함을 무엇으로 감지하는가 — 유입량 하한과 하트비트를 어디에 둘지

#### □ 공공에서는 임계를 정하는 방식에도 제약이 붙음

- 임계를 절대 건수로 잡으면 대도시만 늘 경보가 뜨고 소도시의 상대적 급증은 묻히므로 "평시 분포 대비 몇 배"로 잡아야 형평성이 지켜짐
- 빠른 알림과 정확한 알림 중 무엇을 택했는지, 그 결정을 누가 승인했는지를 알림마다 문서로 남겨야 오발령이 났을 때 설계로 설명할 수 있음

## 7. 핵심 요약

#### □ 1. 스트림은 끝없이 자라는 테이블이고, 배치 질의를 그 위에 그대로 엮는다. (3절)

- 엔진은 중간 결과인 상태를 들고 증분으로 갱신하므로 전체를 다시 세지 않고, trigger가 언제 계산할지를 output mode가 무엇을 내보낼지를 따로 정함
- update 모드가 내보낸 여덟 줄에는 창 22:00-22:10의 강남구가 3과 4로 두 번 들어 있으므로, 받는 쪽은 창과 지역구를 키로 삼은 업서트로 받아야 함

#### □ 2. 창 집계의 기준은 이벤트 시간이고, sliding 건수의 합은 이벤트 수가 아니다. (4절)

- 같은 강남구 7건이 tumbling에서는 합계 7이지만 10분 창·5분 슬라이드에서는 14로 나옴
- 오류가 아니라 정의이므로, 집계표에 창 방식과 크기와 슬라이드를 병기하지 않으면 두 배로 부풀려 읽힘

□ 3. watermark는 관측된 최대 이벤트 시각에서 지연 허용을 뺀 값이다. (5절)

- 시계가 아니라 데이터에서 계산되므로 22:09 → 22:18 → 22:24를 따라 21:59 → 22:08 → 22:14로 전진했고, 유입이 멈추면 창도 닫히지 않음
- 22:04에 발생한 이벤트는 watermark 21:59 시점에 도착해 수용되어 집계 창이 3에서 4로 갱신됐고, 21:49에 발생한 이벤트는 watermark 22:08 시점에 도착해 창이 이미 닫혔으므로 폐기됐음

□ 4. 수용과 폐기는 서로 다른 곳에 증거를 남긴다. (5절)

- 수용은 집계 창의 갱신으로 보이지만 폐기는 결과에 흔적이 없고 폐기 누적 1이라는 카운터로만 확인됨
- 이 카운터를 감시하지 않으면 지연 데이터가 사라진 사실 자체를 알 수 없으며, 4주차의 유실과 같은 구조의 침묵임

□ 5. 알림은 집계 창의 필터지만 미확정·중복·침묵을 함께 설계해야 한다. (6절)

- 지연 수용으로 수치가 자라므로 확정을 기다리는 알림은 최소 지연 허용만큼 늦고, 임계를 3으로 낮추면 같은 창이 2회 발화해 중복 제거 키가 필요함
- 결과 행이 없는 창 둘 중 하나는 이벤트가 없어서이고 하나는 폐기되어서인데 집계만으로는 구분할 수 없으므로, 유입 자체를 보는 감시를 따로 둬

## 8. 과제

□ 실습을 순서대로 실행해 만든 산출물을 LMS에 제출한다

1. 표 5.1 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.
2. `practice/chapter5/data/output/ch5_watermark_trace.json` ( `5-3-watermark-trace.py` 가 만든다)

□ 이어서 본인 산출물의 값을 인용해 네 가지를 각각 세 문장 이내로 쓴다

1. `per_file` 의 세 항목에서 `max_event_time_after` 와 `watermark_after` 를 옮겨 적고, 두 값의 관계를 `watermark_delay_minutes` 로 산술 재현한다.
2. `late_events` 의 두 항목에서 `watermark_at_arrival` 과 `window` 와 `decision` 을 옮겨 적고, 판정이 걸린 이유를 창의 끝과 watermark의 크기 비교로 쓴다.
3. 폐기된 이벤트는 집계 산출물 어디에도 나타나지 않는다. 운영에서 이 폐기를 무엇으로 감지해야 하는지 쓴다.
4. "우리 대시보드 수치는 실시간이라 정확하다"는 문장을 본인의 `window_size_minutes` 와 폐기 판정을 근거로 반박한다.

□ 값을 지어내지 않는다

□ 실행이 안 되면 어디서 막혔는지 그대로 쓴다

□ 수업일 포함 7일 이내에 제출한다

## 9. 더 알아보기

### 스트림 처리에 반드시 Spark를 써야 하는 것은 아니다

- 창·이벤트 시간·watermark는 개념이고 Spark는 구현체 중 하나임. 이 장의 5-3-5-4가 Python 표준 라이브러리만으로 같은 규칙을 재현해 엔진 결과와 일치시킨 것이 그 증거임
- 대표적인 다른 선택지
  - 직접 작성한 소비자 루프 — 4주차의 `kafka-python` 처럼 읽기→처리→커밋을 코드로 뚫. 처리량이 작고 로직이 단순할 때 쓰며, 창·watermark는 직접 구현해야 함
  - **Kafka Streams** — Kafka에 딸린 스트림 처리 라이브러리. 별도 클러스터 없이 애플리케이션 안에 내장되므로 운영 부담이 가장 작음
  - **Apache Flink** — 이벤트 한 건 단위로 처리하는 분산 엔진. micro-batch가 아니어서 지연이 밀리초~초 단위로 작고, watermark 개념은 동일하게 있음
  - **클라우드 관리형** — Google Dataflow, AWS의 Flink 관리형 서비스 등. 엔진 운영을 클라우드가 대신함
- 고르는 기준은 도구의 유행이 아니라 **지연 요구·처리량·상태 관리의 복잡도·팀이 운영할 수 있는 것임**
  - 이 교재가 Spark를 쓰는 이유는 "스트림 = 끝없는 표" 발상 덕에 배치 지식이 그대로 이어지고, 6주차의 배치 처리와 같은 엔진을 쓰기 때문임
- 실습을 더 해 보려면 `practice/chapter5/` 에서 다음을 시도함
  - `5-2-window-assign.py` 의 `SLIDE_MINUTES` 를 2로 바꿔, 건수의 합이 이벤트 수의 몇 배가 되는지 확인함
  - `5-3-watermark-trace.py` 의 `WATERMARK_DELAY_MINUTES` 를 줄여 가며, 몇 분부터 22:04 이벤트가 폐기로 바뀌는지 찾음
  - `5-4-alert-rules.py` 의 `THRESHOLDS` 에 다른 값을 넣어 발화 횟수와 중복 횟수가 어떻게 달라지는지 봄
  - `data/input/stream/` 에 파일을 하나 더 만들어 도착시키면 watermark가 어디까지 전진하는지 확인함
- 뒤 주차에서 이어지는 내용
  - `availableNow` 처럼 쌓인 데이터를 모두 처리하고 끝내는 방식은 6주차의 배치 처리와 만남
  - 폐기 카운터와 유입량 하한을 지표로 감시하는 방법은 11주차에서 다룸
  - 창 집계 결과를 피처로 재사용할 때의 정의 관리는 7주차와 12주차에서 다룸
- 원문을 직접 보고 싶다면 다음 두 편이 출발점임
  - Apache Spark, 「Structured Streaming Programming Guide」 — 창·watermark·output mode의 공식 설명. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
  - Akidau, 「Streaming 101: The world beyond batch」 — 이벤트 시간과 처리 시간을 나누어 보는 관점의 출발점. <https://www.oreilly.com/radar/the-world-beyond-batch-streaming-101/>